

6. Bölüm

Kontrol İşlemleri

6.1 Ardışık İşlem ve Kontrol İşlemleri

Nesne yönelimli programlamada OOP (**object-oriented programming**) yöntemler, yapısal programlamadaki fonksiyonlar gibi kodlanır. Yani ilk önce işlenecek verileri taşıyan veri yapıları tanımlanır ve ardından bu verileri işleyen kontrol yapılarına ilişkin talimatlar yazılır. Şu ana kadar örneğini verdiğimiz kodlarda veri yapısı olarak sadece değişkenler kullanılmıştır. Sonrasında ise giriş çıkış işlemleri talimatlar içeren programlar yazılmıştır. Programın icrası ilk talimattan başlar ve sırasıyla program bitene kadar devam eder. İşte programın icrasını değiştirmeyen bu tür talimatlara **ardışık işlem (sequential operation)** adı verilir. Ardışık işlemler aşağıdaki üç tür talimattan (**statement**) oluşur.

1. Değişkenlerin kimliklendirildiği değişken tanımlamaları (**identifier definition**):
 - (a) `int yariCap=3;`
 - (b) `const float PI=3.14F;`
 - (c) `float daireninAlani,daireninCevresi;`
2. İfadelerden (**expression**): İfade, sabit, değişken ve işlec (**operator**) içeren söz dizimleridir. C#'ta her ifade, mutlaka bir değer döndürür!
 - (a) `daireninAlani=PI*yariCap*yariCap;`
 - (b) `float daireninCevresi=2.0*PI*yariCap;`
3. Klavyeden veri okuma ve konsola veri yazma gibi giriş-çıkış işlemleri (**input-output operation**):
 - (a) `Console.WriteLine("Karekökü Alınacak Sayıyı Giriniz");`
 - (b) `string okunan = Console.ReadLine();`

Kontrol işlemleri (control operation) ise programın icra sırasını değiştiren kontrol yapıları olup bu talimatlar (**statement**) olup üç türü vardır;

1. Duruma göre seçimler (**conditional choice**): `if`, `if-else` ve `switch` talimatları.
2. İlişkisel döngü (**relational loop**): `while`, `do-while` ve `for` talimatları.
3. Dallanmalar (**jump**): `continue`, `break`, `goto` ve `return` talimatları.

Kontrol işlemleri programda yazılan talimatların icra sırasını değiştirir. Bu nedenle icra akışındaki bu değişikliğe **kontrol akışı (control flow)** adı verilir. Kısaca kodun **çalışma sırasıdır**. Kontrol akışı, yazdığınız talimatların hangi koşullarda, hangi döngülerle veya hangi istisnai durumlarda icra edileceğini belirler.

Kontrol işlemlerinde; **kodlanan mantıksal satırlar (logical sequence)** ile fiziksel olarak icra edilen satırlar (**physical sequence**) birbirinden farklıdır.

6.2 Akış Diyagramları

Yazılan programdaki talimatların icra sırasına, **kontrol akışı** demiştik. Kontrol işlemi içeren programların **kontrol akışını anlamak için** akış diyagramları çizilir. Aşağıda akış diyagramlarındaki temel şekiller verilmiştir.

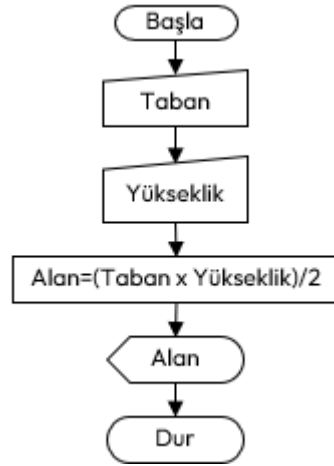


Şekil 6.1: Akış Diyagramları Genel Şekilleri

Akış diyagramları (flowchart), adımların grafik gösterimleridir. Algoritmaları ve programlama mantığını temsil etmek için bir araç olarak bilgisayar biliminden ortaya çıkmıştır. Ancak diğer tüm işlem türlerinde

kullanılmak üzere genişletilmiştir. Günümüzde akış diyagramları, bilgileri göstermede ve akıl yürütmeye yardımcı olmada son derece önemli bir rol oynamaktadır. Karmaşık süreçleri görselleştirmemize veya sorunların ve görevlerin yapısını açık hale getirmemize yardımcı olurlar. Bir akış şeması, uygulanacak bir süreci veya projeyi tanımlamak için de kullanılabilir.

Aşağıda taban ve yüksekliği klavyeden girilen bir dik üçgenin alanını hesaplamak için bir akış diyagramı örneği verilmiştir.



Şekil 6.2: Dik Üçgenin Alana Hesabını Gösteren Akış Diyagramı

Akış diyagramı çizilirken aşağıdaki kurallara uyulur;

- ◊ Akış diyagramlarında tek bir başlangıç simgesi olmalıdır
- ◊ Bitiş simgesi birden çok olabilir.
- ◊ Karar simgesinin haricindeki simgelere her zaman tek giriş ve tek çıkış yolu bulunur.
- ◊ Bağlaç simgesi sayfanın dolmasından ötürü parçalanmış akış şemasının öğelerini birleştirmede kullanılır.
- ◊ Simgeler birbirleri ile tek yönlü okla bağlanırlar.
- ◊ Okların yönü algoritmanın mantıksal işlem akışını tanımlar.

6.3 Duruma Göre Seçimler

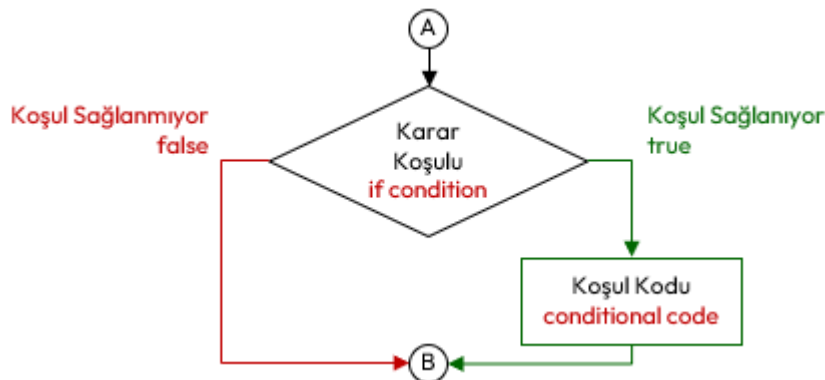
Duruma göre seçimler (**conditional choice**) programın akışını bir koşula göre değiştiren talimatlardır.

§6.3.1 If Talimatı

If talimatı, karar vermeye ilişkin bir talimat olup bir koşula bağlı olarak programın icrasını değiştirir. If talimatının sözde kodu aşağıda verilmiştir;

```
if (karar-kosulu)
    koşul-kodu;
```

Burada ilişkisel işlemler içeren **karar koşulu** (**condition**) ifadesinin (**expression**) sonucu **true** ise **koşul kodu** (**conditional code**) icra edilir, değilse icra edilmez. Bu talimatın akış diyagramı da aşağıdaki gibidir;



Şekil 6.3: If Talimatı İcra Akışı

Aşağıda, yeni nesil konsol uygulamasındaki **Program.cs** içeriğini inceleyelim;

```
1 int yas = 18;
```

```

2 char cinsiyet = 'K';
3 if (yas < 30)
4     Console.WriteLine("Genç");
5 if (cinsiyet == 'E')
6     Console.WriteLine("Erkek");

```

Örnek kodumuzu şu şekilde açıklayabiliriz;

1. ve 2. Satırda `yas` ve `cinsiyet` değişkenleri başlatılmıştır.
3. Satırda bir `if` talimatı bulunmaktadır. **Karar koşulu** olan `yas<30` ifadesinin sonucu `true` olduğundan
4. Satırdaki **koşul kodu** olan `Console.WriteLine("Genç");` talimatı ile konsola "Genç" yazılır.
- Sırada olan 5. Satırdaki kod icra edilir. Bu `if` talimatının karar koşulu olan `cinsiyet=='E'` ifadesinin sonucu `false` olduğundan 6. Satırdaki **koşul kodu** talimatı **icra edilmez**.

If talimatında **koşul kodu**, her zaman yukarıdaki gibi **ardışık işlem** (*sequential operation*) olmaz. Bazen `if` gibi bir başka **kontrol işlemi** (*control operation*) de olabilir. Aşağıda **kademeli if** (*compound if/cascade if*) kullanımına ilişkin kod örneğinde ikinci `if`, birinci `if` talimatının **koşul kodudur**;

```

if (yas<30)
    if (yas<7) // Bu if, üsteki if talimatının koşul kodudur.
        Console.WriteLine("Çocuk");

```

Koşul kodu birden fazla talimattan oluşacak ise kod bloğu içine alınır. Aşağıda verilen kod örneğinde ilk `if` talimatında `yas<30` ile test edilen koşul doğrulandığında kod bloğunun içindeki tüm talimatlar icra edilir.

```

if (yas<30) { //koşul-kodu bloğu başlangıcı
    if (yas<7)
        Console.WriteLine("Çocuk");
    if (yas<18)
        Console.WriteLine("Genç");
    if (yas>60)
        Console.WriteLine("Yaşlı");
} //koşul-kodu bloğu bitişi

```

Bazı durumlarda bir `if`, başka bir `if` bloğu içinde yani **iç içe** (*nested if*) olabilir. Buna ilişkin kod örneği de aşağıda verilmiştir.

```

if (cinsiyet=='E') { // dış if bloğu başlangıcı
    if (yas<7) { // iç if bloğu başlangıcı
        Console.WriteLine("Erkek Çocuk");
        Console.WriteLine("Yada Erkek Bebek");
    } // iç if bloğu başlangıcı
    if (yas<18)
        Console.WriteLine("Genç Delikanlı");
    if (yas>60)
        Console.WriteLine("Yaşlı Adam");
} // dış if bloğu bitişi

```

If talimatında **koşul kodu**, her zaman tek bir test ifadesinden oluşmayabilir. Bahsedilen duruma ilişkin kod örneği de aşağıda verilmiştir.

```

if ((cinsiyet=='E') && (yas<18)) // hem Erkek, hem de genç
    Console.WriteLine("Genç Delikanlı");
if ((cinsiyet=='K') && (yas>60)) // hem yaşlı, hem de kadın
    Console.WriteLine("Yaşlı Kadın");

```

Ayrıca bazen programcı tarafından aşağıdaki gibi `if` talimatları yazabilir.

```

if (true)
    Console.WriteLine("Bu metin konsola her zaman yazılır.");
if (false)
    Console.WriteLine("Bu metin konsola hiçbir zaman yazılmaz!");

```

Örnek olarak klavyeden girilen bir sayının işaretini bulan bir programa ilişkin akış diyagramı Şekil 6.4'de verilmiştir. Şimdi programı yazalım;

```

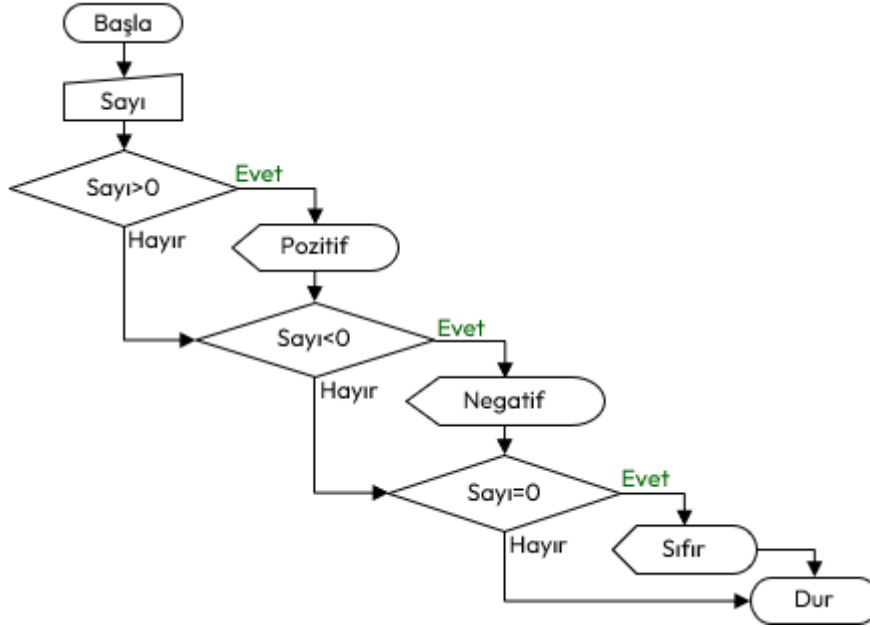
// See https://aka.ms/new-console-template for more information
int sayi;
Console.WriteLine("Sayı Giriniz:");
sayi=Convert.ToInt32(Console.ReadLine());

```

```

if (sayi > 0) // sayının pozitif olup olmadığı test ediliyor.
    Console.WriteLine("Pozitif"); // Burada sayının pozitif olduğu biliniyor.
if (sayi < 0) // sayının negatif olup olmadığı test ediliyor.
    Console.WriteLine("Negatif"); // Burada sayının negatif olduğu biliniyor.
if (sayi == 0) // sayının sıfır olup olmadığı test ediliyor.
    Console.WriteLine("Sıfır"); // Burada sayının sıfır olduğu biliniyor.

```



Şekil 6.4: Bir Sayının İşaretini Bulan Akış Diyagramı

§6.3.2 If-Else Talimatı

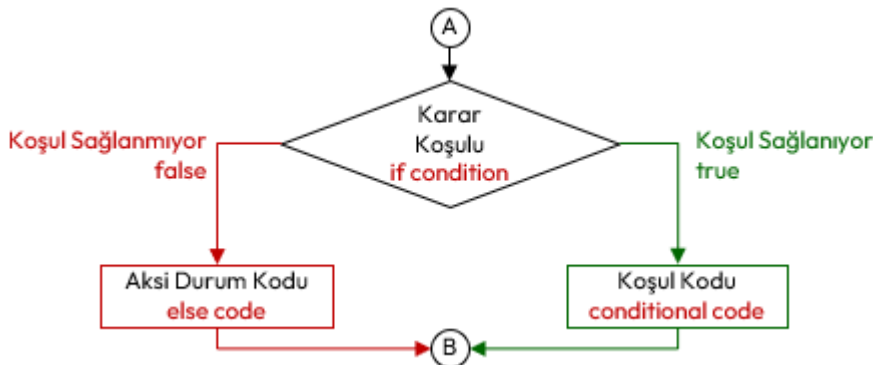
If-Else Talimatı, bir başka karar verme talimatı olup bir koşula bağlı olarak programın icrasını değiştirir. Sözde kodu aşağıda verilmiştir;

```

if (karar-kosulu)
    kosul-kodu;
else
    aksi-durum-kodu;

```

If-Else talimatında, **karar koşulu** (condition) ifadesi **true** ise **koşul kodu** (conditional code) icra edilir, karar koşulu **false** ise **aksi durum kodu** (else code) icra edilir.



Şekil 6.5: If-Else Talimatı İcra Akışı

Aşağıda verilen örnek programı inceleyelim;

```

1 int yas = 65;
2 char cinsiyet = 'K';
3 if (yas < 50)
4     Console.WriteLine("Genç");
5 else
6     Console.WriteLine("Yaşlı");

```

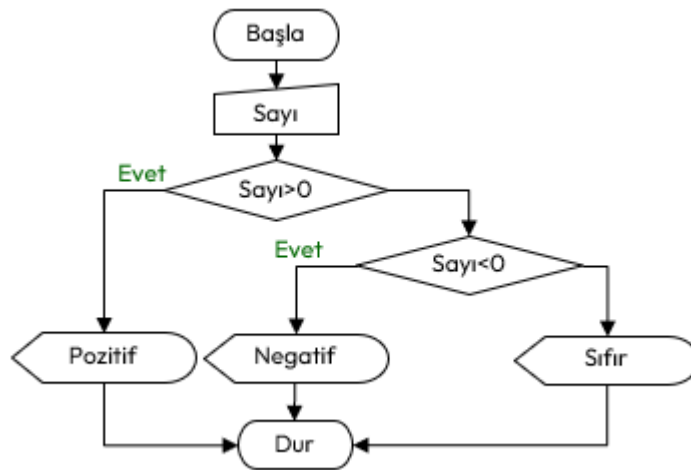
Örnek kodumuzu şu şekilde açıklayabiliriz;

1. ve 2. Satırda `yas` ve `cinsiyet` değişkenleri başlatılmıştır.
3. Satırda bir `if` talimatı bulunmaktadır. **Koşul kodu** olan `yas<50` ifadesinin sonucu `false` olduğundan
4. Satırdaki **koşul kodu** talimatı **icra edilmez**. Bunun yerine 5. satırdan başlayan `else` kodu yani 6. Satırdaki **aksi durum kodu** icra edilir. Yani `Console.WriteLine("Yaşlı");` talimatı ile konsola "Yaşlı" yazılır.

If talimatında olduğu gibi bu talimatta da **koşul kodu** ya da **aksi durum kodu** birden fazla talimattan oluşacak ise kod bloğu içine alınır. Buna ilişkin kod örneği aşağıda verilmiştir.

```
if (cinsiyet=='E') {
    if (yas<7)
        Console.WriteLine("Erkek Çocuk");
    if (yas<18)
        Console.WriteLine("Genç Delikanlı");
    if (yas>60)
        Console.WriteLine("Yaşlı Adam");
} else {
    if (yas<3)
        Console.WriteLine("Kız Bebek");
    if (yas<7)
        Console.WriteLine("Kız Çocuk");
    if (yas<18)
        Console.WriteLine("Genç Kız");
    if (yas>60)
        Console.WriteLine("Yaşlı Kadın");
}
```

Bir sayının işaretini bulan programının icra akışı olan Şekil 6.4 incelendiğinde ilk `if` talimatında yapılan test doğru çıkarsa geri kalan testlerin yapılmasına gerek kalmadığı anlaşılabılır. Benzer şekilde ilk iki test geçilirse üçüncü `if` talimatındaki teste gerek yoktur. Bu durumu Şekil 6.6'da verilen akış diyagramında daha net görebiliriz. Bu durumda aynı program If-Else talimatlarıyla aşağıdaki şekilde yeniden yazabiliriz;



Şekil 6.6: Bir Sayının İşaretini Bulan If-Else Akış Diyagramı

```
1 // See https://aka.ms/new-console-template for more information
2 int sayi;
3 Console.WriteLine("Sayı Giriniz:");
4 sayi=Convert.ToInt32(Console.ReadLine());
5 if (sayi > 0)
6 { // sayının pozitif olup olmadığı test ediliyor.
7     Console.WriteLine("Pozitif"); // Burada sayının pozitif olduğu biliniyor.
8 }
9 else
10 { // Burada sayının pozitif olmadığı biliniyor.
11     if (sayi < 0)
12     { // sayının negatif olup olmadığı test ediliyor.
13         Console.WriteLine("Negatif"); // Burada sayının negatif olduğu biliniyor.
```

```

14     }
15     else
16     { // Burada sayının pozitif ve negatif olmadığı test biliniyor.
17         Console.WriteLine("Sıfır"); // Burada sayının sıfır olduğu biliniyor.
18     }
19 }

```

Örnek kodumuzu şu şekilde açıklayabiliriz;

- ◊ Eğer `sayı` pozitif girilirse;
 - 5. Satırdaki `if` karar koşulu olan `sayı>0` ifadesinin sonucu `true` çıkacak ve sadece 7. Satırdaki koşul kodu olan `Console.WriteLine("Pozitif");` talimatı icra edilecektir. 9. Satırdan başlayan ve 19. satırda biten aksi durum kodu icra edilmeyecektir.
- ◊ Eğer `sayı` negatif girilirse;
 - 5. Satırdaki `if` karar koşulu olan `sayı>0` ifadesinin sonucu `false` çıkacak ve 9. Satırdan başlayan ve 19. Satırda biten aksi durum kodu icra edilecektir. Bu blok içinde
 - 11. Satırdaki `if` karar koşulu olan `sayı<0` ifadesinin `true` çıkacak ve 13. Satırdaki koşul kodu olan `Console.WriteLine("Negatif");` talimatı icra edilecektir.
- ◊ Eğer `sayı` sıfır girilirse;
 - 5. Satırdaki `if` karar koşulu olan `sayı>0` ifadesinin sonucu `false` çıkacak ve 9. Satırdan başlayan ve 19. Satırda biten aksi durum kodu icra edilecektir. Bu blok içinde;
 - 11. Satırdaki `if` karar koşulu olan `sayı<0` ifadesinin sonucu `false` çıkacak ve 13. Satırdaki koşul kodu talimatı icra edilemeyecektir. Onun yerine on 15. Satırdan başlayan aksi durum kodu icra edilecektir. Yani 17. Satırdaki `Console.WriteLine("Sıfır");` icra edilecektir.

Ayrıca kod incelendiğinde her bir `if` ve `else` sonrasında tek talimat bulunmaktadır. Dolayısıyla blok kullanmaya gerek de yoktur. Bu durumda kodun nihai hali aşağıda verilmiştir.

```

int sayi;
Console.WriteLine("Sayi Giriniz:");
sayi=Convert.ToInt32(Console.ReadLine());
if (sayi>0)
    Console.WriteLine("Pozitif");
else if (sayi<0)
    Console.WriteLine("Negatif");
else
    Console.WriteLine("Sıfır");

```

§6.3.3 Sarkan Else

Aşağıdaki program örneği incelendiğinde `else`, hangi `if` talimatına aittir? Böyle bir kod programcı tarafından yazılabilir ve derleyici buna hiçbir sıkıntı çıkarmaz. Bu problem **sarkan else** (**dangling else**) problemi olarak bilinir.

```
if (cinsiyet=='E') if (yas<18) Console.WriteLine("Delikanlı"); else Console.WriteLine("Adam");
```

Bu durumda kodu aşağıdaki gibi okunaklı hale getirdiğimizde ilk `if` talimatının **koşul kodunun** ikinci `if-else` talimatı olduğu görülmektedir.

```

if (cinsiyet=='E')
    if (yas<18)
        Console.WriteLine("Delikanlı");
    else
        Console.WriteLine("Adam");

```

Kod bloğu kullanılmadan yazılan bu tip kodlarda `else` her zaman kendinden bir önceki `if` talimatına aittir.

Sarkan else durumundaki mantıksal hataların önüne geçmek için
acemi kullanıcılar her zaman blok ve girinti kullanmalıdır.

```

if (cinsiyet=='E') {
    if (yas<18) {
        Console.WriteLine("Delikanlı");
    } else {
        Console.WriteLine("Adam");
    }
}

```

```

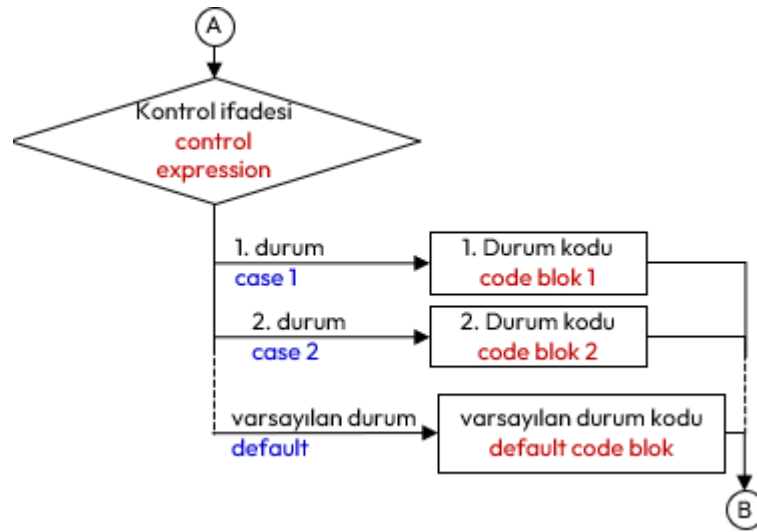
}
```

§6.3.4 Switch Talimatı ve İfadesi

Switch talimatı, bir kontrol ifadesi (**control expression**) sonucunda birden fazla alternatif arasında seçim yapılmasını sağlar. Sözde kodu aşağıda verilmiştir;

```

switch (kontrolifadesi) { //switch bloğu başlangıcı
    case alternatif-1:
        alternatif-1-kodu;
        break; //break talimatı zorunlu değildir
    case alternatif-2:
        alternatif-2-kodu;
        break; //break talimatı zorunlu değildir.
        // ...
    default: // Bu etiket zorunlu değildir!
        varsayılan-alternatif-kodu;
} //switch bloğu bitişi
```



Şekil 6.7: Switch Talimatı İcra Akışı

Switch talimatına ilişkin kurallar;

1. **Kontrol ifadesi**, bir kez değerlendirilir. Kontrol ifadesi temel sayı tipleri olabileceği gibi **string** de olabilir.
2. **Kod bloğu olmayan bir switch talimatı yazılamaz!**
3. **Her bir alternatif, kontrol ifadesinin sonuçlarından biri olup izleyen ve iki nokta ile biten bir etiket (label) olarak tanımlanır.**
4. **Alternatif, hiçbir zaman değişken veya ifade (expression) olamaz!**
5. **switch bloğu icra edilirken**, karşılaştığı ilk **break;** talimatında, **switch** bloğu dışına çıkılır. Bu talimat zorunlu değildir.
6. Tüm alternatifleri içerecek şekilde etiketler yazılmayabilir. Blok sonunda yer alacak şekilde **üzerinde yer alan alternatifler dışında kalan tüm alternatifler için default:** etiketli alternatif yazılabilir. Bu etiket de zorunlu değildir.

Aşağıda klavyeden girilen bir sayının 4 rakamına bölünmesinde kalan üzerinden işlem yapan **switch** talimatı bir başka örnek program verilmiştir;

```

1 // See https://aka.ms/new-console-template for more information
2 int sayi;
3 Console.WriteLine("Bir Sayı Giriniz:");
4 sayi=Convert.ToInt32(Console.ReadLine());
5 switch (sayi % 4)
6 { //kontrol ifadesi bir işlem içerir
7 case 3:
8     Console.WriteLine("Sayı 4 rakamına bölündüğünde kalan 3 dir.");
9     break;
```

```

10 case 2:
11 case 1:
12     Console.WriteLine("Sayı 4 rakamına bölündüğünde kalan 2 veya 1 dir.");
13     break;
14 default: // Başka alternatif yoktur.
15     Console.WriteLine("Sayı 4 Rakamına TAM Bölünür");
16     break;
17 }

```

Örnek kodumuzu şu şekilde açıklayabiliriz;

- ◊ Eğer `menu, 1,5,9,...` girilirse;
 - 5. Satırdaki `switch` talimatında kontrol ifadesinin sonucu 1 olacaktır.
 - Bu durumda 11. Satırdaki `case 1:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
 - İlk önce 12. Satırdaki `Console.WriteLine("Sayı 4 rakamına bölündüğünde kalan 2 veya 1 dir.");` icra edilir ve bir sonraki talimata geçilir.
 - Bir sonraki talimat 13. Satırdaki `break;` talimatı bizi `switch` bloğunun dışına çıkarır.
- ◊ Eğer `menu, 2,6,10,...` girilirse;
 - 5. Satırdaki `switch` talimatında kontrol ifadesinin sonucu 2 olacaktır.
 - Bu durumda 10. Satırdaki `case 2:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
 - Bulunan ilk talimat 12. satırdaki talimattır.
 - İlk önce 12. Satırdaki `Console.WriteLine("Sayı 4 rakamına bölündüğünde kalan 2 veya 1 dir.");` icra edilir ve bir sonraki talimata geçilir.
 - Bir sonraki talimat 13. Satırdaki `break;` talimatı bizi `switch` bloğunun dışına çıkarır.
- ◊ Eğer `menu, 3,7,11,...` girilirse;
 - 5. Satırdaki `switch` talimatında kontrol ifadesinin sonucu 3 olacaktır.
 - Bu durumda 7. Satırdaki `case 3:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
 - İlk önce 8. Satırdaki `Console.WriteLine("Sayı 4 rakamına bölündüğünde kalan 3 dir.");` icra edilir ve bir sonraki talimata geçilir.
 - Bir sonraki talimat 9. Satırdaki `break;` talimatı bizi `switch` bloğunun dışına çıkarır.
- ◊ Eğer `menu, 4,8,12,...` girilirse;
 - 5. Satırdaki `switch` talimatında kontrol ifadesinin sonucu 0 olacaktır.
 - Bu durumda diğer etiketlerde olmayan bir durum olan 14. Satırdaki `default:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
 - İlk önce 15. Satırdaki `Console.WriteLine("Sayı 4 Rakamına TAM Bölünür");` icra edilir ve bir sonraki talimata geçilir.
 - Daha sonra 16. Satırdaki `break;` talimatı bizi `switch` bloğunun sonundan dışına çıkarır.

Şu ana kadar anlatılan Switch talimatı, **sabit desen (constant pattern)** kullanan talimattır. En klasik `switch` kullanımıdır. Bir değişkenin değerinin, önceden belirlenmiş bir **değişmez (literal)** değere eşit olup olmadığını kontrol eder. Aşağıda `string` ile bu testleri yapan bir program örneği verilmiştir;

```

string veri = "C#";
switch (veri) {
case "Java":
    Console.WriteLine("Dil: Java");
    break;
case "C#": // Sabit desen: Değer tam olarak "C#" mı?
    Console.WriteLine("Dil: C#");
    break;
case null: // Sabit desen: Veri null mı?
    Console.WriteLine("Veri yok.");
    break;
default:
    Console.WriteLine("Bilinmeyen dil.");
    break;
}

```

Yukarıdaki kod bloğunda da aslında büyük bir sorun diyebileceğimiz bir şey yoktur fakat, `case` sayısı arttıkça, bu blok uzayacak ve aslında kodda kalabalık yapacak `case` ve `break` anahtar kelimeleri de artacaktır. Sadece string atama yaptığımız bir `switch` bloğunu aşağıdaki gibi daha sade bir hale **switch ifadeleri (switch**

expression) ile getirebiliriz.

C#9+ ile gelen **switch ifadelerindeki ilişkisel desen (relational pattern)** özelliği ile, **case** etiketleri ifade (**expression**) olarak işlem görmeye başlamıştır. **Case ifadeleri** içinde doğrudan büyüklük/küçüklük (<, >, <=, >=) işleçlerini kullanmanıza olanak tanır.

```
int sicaklik = 25;
string durum = sicaklik switch
{
    < 0      => "Donuyor",          // İlişkisel desen
    >= 0 and <= 15 => "Soğuk",      // Mantıksal ve ilişkisel desen bir arada
    > 15 and <= 25 => "Ilık",
    > 25      => "Sıcak",
    _        => "Geçersiz veri"    // Discard (Geri kalan her şey)
};
Console.WriteLine($"Hava durumu: {durum}");
```

Ayrıca switch ifadelerinde bir de **mantıksal desenler (logical latterns)** kullanılmaya başlamıştır. Yukarıda bahsedilen desenleri birleştirirken **and**, **or** ve **not** anahtar kelimelerini kullanabiliriz. Bu, okunabilirliği zirveye taşır.

```
object girdi = 45;
if (girdi is int sayi and > 0) // Hem tip kontrolü hem ilişkisel desen
{
    Console.WriteLine("Pozitif bir tam sayı girildi.");
}
// switch içinde kullanımı:
string sonuc = girdi switch
{
    not int => "Tam sayı değil",
    < 0 or > 100 => "Sınır dışında",
    >= 0 and <= 100 => "Geçerli puan",
    _ => "Bilinmiyor"
};
```

§6.3.5 Üçlü Koşul İşleci

Daha önce 4.10.3 başlığı altında işlenen işleçlere ek olarak, **ardışık işlemlerde (sequential operation)** if talimatlarına gerek kalmadan bir koşula göre **ifadeler (expression)** işlenecek ise **üçlü koşul işleci (conditional tenary operator)** kullanılır. Aşağıda üçlü işlecin iki söz dizimi verilmiştir;

koşul ? doğruifadesi : yanlışifadesi;
değişken = koşul ? doğruifadesi : yanlışifadesi;

Aşağıda oy kullanma durumuna ilişkin örnek işlenmiştir;

```
int yas;
Console.WriteLine("Yaşınız?: ");
yas=Convert.ToInt32(Console.ReadLine());
string metin=(yas >= 18) ? "Oy Kullanabilirsin." : "Oy Kullanamazsın!";
Console.WriteLine(metin);
```

Aşağıda başka kullanımlarına örnek verilmiştir;

```
int sayı;
bool tek, çift;
Console.WriteLine("Bir Sayı Giriniz: ");
sayı=Convert.ToInt32(Console.ReadLine());
tek = (sayı % 2==0) ? true : false;
çift =!tek;
int sonuç1 = tek ? sayı + 30 : sayı + 40;
int sonuç2 = çift ? sayı * 30 : sayı * 40;
Console.WriteLine($"Sonuç1: {sonuç1}, Sonuç2: {sonuç2}");
```

6.4 İlişkisel Döngüler

İşlemciler iyi bir sayıacıdır. CPU içindeki kaydediciler (**registers**) sayma işlevini ve birçok matematiksel işlemi yapmak için de kullanılır. **Birçok problemlerin çözümüne, belli adımların tekrarlanması ile gidilir.** Bu tip problemler şimdiye kadar olan yöntemlerle yapılırsa, tekrar tekrar aynı kodu yazmak zorunda kalırız.

Konsola 10 defa isminizi yazdıran bir program örneğini ele alalım.

```
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
```

Program incelendiğinde aynı `WriteLine` talimatının tekrar tekrar yazıldığını görürüz. **Kod tekrarı istenmeyen bir durumdur!**

§6.4.1 Sayaç Kontrollü Döngüler

Belirlenen bir sayaç değişkeninin istenilen bir değere ulaşmış olup olmadığını kontrol ederek kodun tekrar tekrar icra edilmesi sağlanır. Yapısal programlama öncesinde tekrar edilecek kodun başına dönüş `goto` talimatıyla sağlanır. **Bu talimat icra sırasını etkileyen bir talimattır.** Kodlama sırasında tekrar edilecek kodun başına iki



Şekil 6.8: Sayaç Kontrollü Döngü İcra Akışı

nokta ile biten bir **etiket (label)** konulur. Etiketlere kimlik verilirken yine 4.2.5 başlığındaki değişken kimliklendirme (**identifier definition**) kuralları uygulanır. Tekrar edilecek kodun sonunda ise `goto` talimatı etiketle birlikte kullanılır.

```
1 // See https://aka.ms/new-console-template for more information
2 int sayaç=0;
3 Etiket:
4 Console.WriteLine("İLHAN");
5 sayaç++;
6 if(sayaç<10)
7     goto Etiket;
```

Örneği şu şekilde açıklayabiliriz;

- ◇ 2.Satırda tekrar edilecek koda girmeden sayacıma ile değer verdik.
- ◇ 3.Satıra geri dönülecek kodun başını işaretleyen **Etiket:** kimlikli bir etiket konulmuştur.
- ◇ 4.Satırda tekrarlanacak talimat yazılmıştır. Birden fazla talimat da yazabiliriz.
- ◇ 5.Satırda sayacıma güncelledik.
- ◇ 6.Satırda sayacın istenilen değere ulaşmış olup olmadığını kontrol ettik. Eğer ulaşmamış ise 7.Satırdaki `goto Etiket;` talimatıyla 3.Satıra döndük.

Sonuç olarak 3. ile 7. Satır arasında dört talimat içeren **mantıksal satır (logical sequence)** olmasına karşın 40

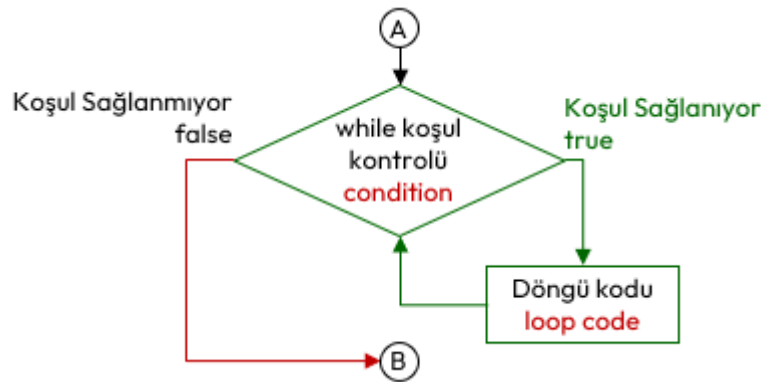
adet **fiziksel satır** (**physical sequence**) icra edilmiştir. C# programlama dili tarafından **goto** talimatı desteklenir. Ancak 1968 Yılında *Edsger W. Dijkstra* tarafından **goto** ifadelerinin zararlı olduğunu ilan ettiği unutulmamalıdır.

Yukarıda verilen döngü kodu örneğindeki gibi arapsaçı kodlamadaki **goto** talimatı yerine, **yapısal programlamada** (**structural programming**) ilişkisel döngü (**relational loop**) talimatları olan **while**, **do-while** ve **for** talimatları geliştirilmiştir.
goto talimatının kullanımı zorunlu olmadıkça önerilmez!

§6.4.2 While Talimatı

While döngüsünde **koşul** (**condition**), **true** olduğu sürece **döngü kodu** (**loop code**) icra edilir. Tekrarlanacak kod, tek bir talimat olabileceği gibi bir kod bloğu da olabilir. While döngüsünün sözde kodu aşağıda verilmiştir;

```
while (koşul)
    döngü-kodu;
```



Şekil 6.9: While Talimatı İcra Akışı

While talimatı kullanılarak 6.4.1 başlığında verilen döngü örneği, aşağıdaki şekilde yazılabilir;

```
int sayaç=0; //sayaça ilk Değer 0 Olarak Verildi
while (sayaç<10) // while koşul kontrolü
{ // döngü bloğu
    Console.WriteLine("İLHAN"); // Döngüde çalıştırılacak kod.
    sayaç=sayaç+1; //Sayaç Güncelleme
}
```

Sayaç güncelleme ifadesi, while **koşul** ifadesine eklenerek program daha da kısaltılabilir. Her koşul kontrolü sonrası sayaç artırılacaktır;

```
int sayaç=0; //sayaça ilk Değer 0 Olarak Verildi
while (sayaç++<10) // while koşul kontrolü
    Console.WriteLine("İLHAN"); // Döngüde çalıştırılacak kod.
```

Yukarıdaki örnek incelendiğinde döngü bloğu bir adet talimat içeren mantıksal satır (**logical sequence**) içermesine rağmen 10 adet fiziksel satır (**physical sequence**) icra edilmiştir.

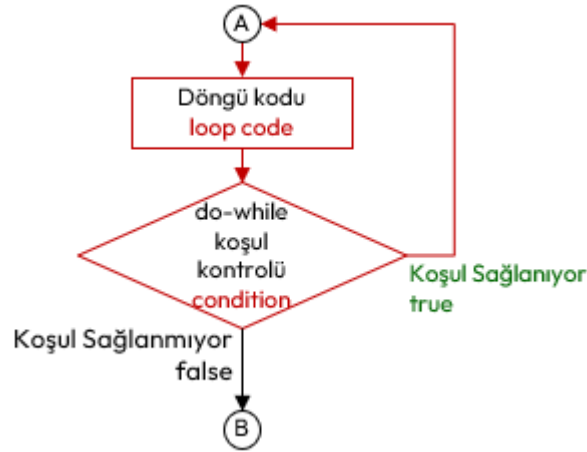
§6.4.3 Do-While Talimatı

Do-While döngüsünde, **do** ile **while** saklı kelimeleri arasındaki **döngü kodu** (**loop code**), **koşul** (**condition**) **true** olduğu sürece tekrarlanarak icra edilir. Tekrarlanacak kod tek bir talimat olabileceği gibi bir kod bloğu da olabilir. Do-While talimatının sözde kodu aşağıda verilmiştir;

```
do // "do" kelimesi goto talimatındaki etiket gibi davranır.
    döngü-kodu;
while (koşul);
```

Do-While talimatı kullanılarak 6.4.1 başlığında verilen döngü örneği, aşağıdaki şekilde yazılabilir;

```
int sayaç=0; //sayaça ilk Değer 0 Olarak Verildi
do
{ // döngü bloğu
    Console.WriteLine("İLHAN"); // Döngüde çalıştırılacak kod.
    sayaç=sayaç+1; //Sayaç Güncelleme
}
```



Şekil 6.10: Do-While Talimatı İcra Akışı

```
} while (sayaç<10); // Do-While koşul kontrolü
```

Sayaç güncelleme ifadesi, **koşul** ifadesine eklenerek program aşağıdaki şekilde daha da kısaltılabilir.

```
int sayaç=0; //sayaça ilk Değer 0 Olarak Verildi
do
    Console.WriteLine("İLHAN"); // Döngüde çalıştırılacak kod.
while (sayaç++<10); // Do-While koşul kontrolü
```

Yukarıdaki son örnek incelendiğinde ibir adet talimat içeren mantıksal satır (**logical sequence**) içermesine rağmen 10 adet fiziksel satır (**physical sequence**) icra edilmiştir.

While döngüsünde koşul kontrolü en başta yapılır!
Do-While döngüsünde ise döngü bloğu en az bir kez çalıştırıldıktan sonra koşul kontrolü yapılır!

§6.4.4 For Talimatı

For döngüsü özellikle tekrar edilen işlemlerin sayısı belli olduğunda kullanılan bir döngü yapısıdır. Sayaç kontrollü döngüler için en iyi seçimdir. Sözde kodu aşağıda verilmiştir.

```
for (ilk-değer-verme; koşul-kontrolü; sayaç-güncelleme)
    döngü-kodu;
```

For talimatı aşağıdaki şekilde icra edilir;

1. **İlk değer verme ifadesi bir kez icra edilir!**
2. **Koşul kontrolü yapılır!** Eğer test sonucu **true** ise döngü kodu icrasına geçilir. Aksi halde döngüden çıkarılır.
3. **Döngü kodu** icra edilir.
4. Döngü kodu icrası bitince **sayaç güncellemeleri yapılır.**
5. Tekrar ikinci adımdaki **koşul kontrolüne** dönülür.

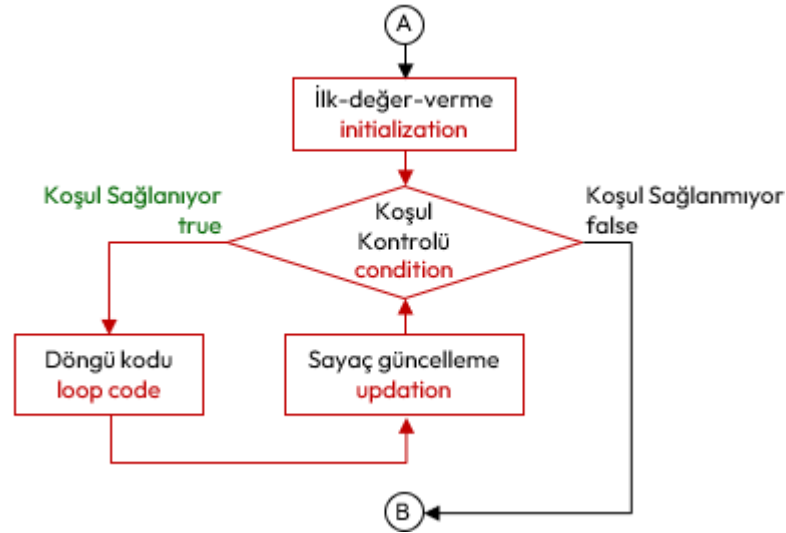
For talimatı kullanılarak 6.4.1 başlığında verilen döngü örneği, aşağıdaki şekilde yazılabilir.

```
for (int sayaç=0; sayaç<10; sayaç++)
    Console.WriteLine("İLHAN");
```

Buradaki döngü mantıksal satır (**logical sequence**) olarak iki talimattan oluşmaktadır. Ancak fiziksel satır (**physical sequence**) olarak 10 satır icra edilmiştir.

Aşağıda konsola yazdığımız her bir satırın başına satır numarası koyan bir program örneği gösterilmektedir.

```
1 // See https://aka.ms/new-console-template for more information
2 for (int sayaç=0; sayaç<10; sayaç++)
3     Console.WriteLine($"{sayaç}-İLHAN");
4 /*Program Çıktısı:
5 0-İLHAN
6 1-İLHAN
7 2-İLHAN
```



Şekil 6.11: For Talimatı İcra Akışı

```

8  3-İLHAN
9  4-İLHAN
10 5-İLHAN
11 6-İLHAN
12 7-İLHAN
13 8-İLHAN
14 9-İLHAN
15 */

```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu 2. Satırda belirtilen bir adet talimat içermektedir. Mantıksal olarak 1 satır içermesine rağmen 10 adet fiziksel satır icra edilmiştir.

For döngüsü, bir sayaç ile çalışılabileceği gibi **birden fazla sayaç ile de çalışılabilir**. Hem ilk değer verme hem de sayaç güncellemede birden fazla sayaca ilişkin işlem yapılabilir. Bunun için **virgül işlecisi (comma operator)** kullanılır.

```

for (int artanSayaç=0, azalanSayaç=10; artanSayaç<10; artanSayaç++, azalanSayaç--)
    Console.WriteLine($"{artanSayaç}-İLHAN-{azalanSayaç}");
/*Program Çıktısı:
0-İLHAN-10
1-İLHAN-9
2-İLHAN-8
3-İLHAN-7
4-İLHAN-6
5-İLHAN-5
6-İLHAN-4
7-İLHAN-3
8-İLHAN-2
9-İLHAN-1
*/

```

Aşağıda klavyeden girilen iki sayı aralığında tek ve çift sayıların toplamını bulan ve ekrana yazan programı verilmiştir.

```

int sayaç, say1,say2;
int tekToplam=0,çiftToplam=0;
Console.WriteLine("Birinci Sayıyı Giriniz: ");
say1=Convert.ToInt32(Console.ReadLine());
Console.WriteLine("İkinci Sayıyı Giriniz: ");
say2=Convert.ToInt32(Console.ReadLine());
if (say12<say1) { //say12, say1 den büyük olmalı.
    int temp=say1; //küçük ise yer değiştiriyoruz.
    say1=say2;
    say2=temp;
}

```

```

for (sayac=say1; sayac<=say12; sayac=sayac+1) {
    if (sayac%2==1) //sayac tek mi?
        tekToplam+=sayac;
    else
        çiftToplam+=sayac;
}
Console.WriteLine($"Tek Toplam:{tekToplam} Çift Toplam:{çiftToplam}");
/* Program Çıktısı:
Birinci Sayıyı Giriniz:
11
İkinci Sayıyı Giriniz:
1
Tek Toplam:36 Çift Toplam:30
*/

```

Bir ve kendisinden başka tam böleni olmayan sayıya **asal sayı** denir. Klavyeden girilen pozitif bir tam sayının asal olup olmadığını ekrana yazdıran program aşağıdaki şekilde kodlanabilir;

```

int sayı;
bool asalmı = true; // C#'ta mantıksal durumlar için bool kullanımı tercih edilir.
Console.Write("Bir Sayı Giriniz: ");
// Kullanıcı girişini okuyup tam sayıya dönüştürüyoruz.
if (int.TryParse(Console.ReadLine(), out sayı)) {
    if (sayı <= 1)
        asalmı = false;
    else {
        // Döngü: Sayıdan bir eksik değerden başlayıp 2'ye kadar iner.
        // Eğer asalmı false olursa (veya operatörü), döngü erken sonlanır.
        for (int sayac = sayı - 1; (sayac > 1) && (asalmı == true); sayac--) {
            if (sayı % sayac == 0)
                asalmı = false; // Sayı bölündüğü an asal değildir.
        }
    }
    // Sonucu ekrana yazdırıyoruz.
    if (asalmı)
        Console.WriteLine($"Girilen {sayı} sayısı asaldır.");
    else
        Console.WriteLine($"Girilen {sayı} sayısı asal değildir.");
} else
    Console.WriteLine("Geçersiz bir sayı girdiniz.");

```

Klavyeden girilen pozitif bir tam sayının **asal çarpanlarını** ekrana yazdıran program aşağıdaki şekilde kodlanabilir;

```

int sayı;
Console.Write("Bir Sayı Giriniz: ");
// Konsoldan okunan metni sayıya dönüştürüyoruz
if (int.TryParse(Console.ReadLine(), out sayı)) {
    // Döngü: Girilen sayıdan başlayıp 1'e kadar iner
    for (int sayac = sayı; sayac > 0; sayac--) {
        // Sayı sayaca tam bölünüyor mu?
        if (sayı % sayac == 0)
            Console.WriteLine($"Bölen: {sayac}");
    }
} else
    Console.WriteLine("Lütfen geçerli bir tam sayı giriniz.");
/* Program Çıktısı:
Bir Sayı Giriniz:12
Asal çarpan:12
Asal çarpan:6
Asal çarpan:4
Asal çarpan:3
Asal çarpan:2
Asal çarpan:1
*/

```

```
*/
```

§6.4.5 İç İçe Döngü Kodu

Döngüler içinde yer alan döngü kodu bir başka döngüyü içerebilir. Örneğin ekrana satır ve sütun içeren bir şekil oluşturmak istediğimizde **iç içe döngü** (**nested loop**) kullanırız. Aşağıda buna ilişkin bir örnek verilmiştir. İlk **for** döngüsünün tekrarlanan kodu ikinci bir **for** döngüsüdür.

```
int satır,sütun;
for (satır=0; satır<5; satır++)
{
    for (sütun=0; sütun<4; sütun++)
        Console.WriteLine("*");
}
```

```
/* Program Çıktısı:
```

```
****
****
****
****
****
*/
```

Bir başka örnek olarak elemanları, satır ve sütun çarpımları olan 4x5 boyutlarındaki matrisi ekrana yazdıran program aşağıda verilmiştir;

```
int satır, sütun;
// 1. Başlığı Yazdıran Kısım
Console.WriteLine(" Sütun: ");
for (sütun = 0; sütun < 3; sütun++)
// {0,3} ifadesi: 0. indeksteki veriyi 3 karakter genişliğinde sağa yasla demektir.
Console.WriteLine($"{(sütun + 1),3} ");
Console.WriteLine();
// 2. Matrisi Yazdıran Kısım
for (satır = 0; satır < 5; satır++) {
    // D2: Onluk sistemde 2 hane olacak şekilde yaz (01, 02 gibi).
    // PadLeft(2, '0') alternatifi de kullanılabilir.
    Console.WriteLine($"{(satır + 1):D2}.satır: ");
    for (sütun = 0; sütun < 3; sütun++)
        // Her sütun değerini 3 karakter genişliğinde ve başına 0 ekleyerek yaz.
        Console.WriteLine($"{sütun:D3} ");
    // Satır bitince imleci alt satıra geçir.
    Console.WriteLine();
}
```

```
/* Program Çıktısı:
```

```
Sütun: 1 2 3
01.satır: 000 001 002
02.satır: 000 001 002
03.satır: 000 001 002
04.satır: 000 001 002
05.satır: 000 001 002
*/
```

§6.4.6 Gözcü Kontrollü Döngüler

Döngüler her zaman bir sayaca bağlı çalıştırılmaz. Bir döngüden çıkış koşulu döngü kodu içerisinde üretilen bir değere bağlı ise bu değere **gözcü değeri** (**sentinel value**) adı verilir. Buradaki gözcü değeri beklenen bir değer dışındaki bir değerdir. Buna örnek olarak klavyeden 0 girilene kadar girilen rakamları toplayan bir program verilmiştir.

```
int sayı; /* okunan tamsayı */
int toplam = 0; /* girilen sayıların toplamı. başlangıçta 0*/
do
{
    Console.WriteLine("Bir sayı giriniz (Bitirmek için 0): ");
    sayı=Convert.ToInt32(Console.ReadLine());
```

```

    toplam += sayı; // sayı 0 girilse bile toplam etkilenmez
} while (sayı != 0); //sentinel value 0
Console.WriteLine($"Girilen Sayıların Toplamı: {toplam}");
/*Program Çıkışı:
Bir sayı giriniz (Bitirmek için 0):
10
Bir sayı giriniz (Bitirmek için 0):
20
Bir sayı giriniz (Bitirmek için 0):
30
Bir sayı giriniz (Bitirmek için 0):
5
Bir sayı giriniz (Bitirmek için 0):
0
Girilen Sayıların Toplamı: 65
*/

```

Aşağıda pozitif sayı girilmesini sağlayan bir başka kod örneği verilmiştir.

```

int sayı;
int pozitifSayı = 0;
do {
    Console.Write("Lütfen pozitif bir tam sayı giriniz: ");
    bool geçerliMi = int.TryParse(Console.ReadLine(), out sayı);
    // 2. İyileştirme: Hem sayısal geçerliliği hem de pozitiflik şartını kontrol ediyoruz.
    if (!geçerliMi || sayı <= 0)
        Console.WriteLine("HATA: Geçersiz giriş! Lütfen pozitif bir tam sayı giriniz.");
} while (sayı <= 0);
pozitifSayı = sayı;
Console.WriteLine($"Girilen pozitif tam sayı: {pozitifSayı}");

```

Aynı örnek bir başka şekilde aşağıdaki gibi kodlanabilir. Ancak burada 4. ve 5. satırlar ile 10. ve 11. satırların aynı olduğuna ve kod tekrarı yapıldığına dikkat ediniz!

```

1 int sayı = 0;
2 int pozitifSayı;
3 // Kullanıcıdan ilk girişi alıyoruz (Döngüye girebilmek için şartın kontrolü lazım)
4 Console.Write("Lütfen pozitif bir tam sayı giriniz: ");
5 bool basarili = int.TryParse(Console.ReadLine(), out sayı);
6 // Şart: Sayı geçersizse (harf vb.) VEYA sayı <= 0 ise döngüye gir
7 while (!basarili || sayı <= 0)
8 {
9     Console.WriteLine("HATA: Pozitif bir tam sayı girmediniz!");
10    Console.Write("Lütfen pozitif bir tam sayı giriniz: ");
11    basarili = int.TryParse(Console.ReadLine(), out sayı);
12 }
13 pozitifSayı = sayı;
14 Console.WriteLine($"Girilen pozitif tam sayı: {pozitifSayı}");

```

§6.4.7 BigInteger Veri Tipi

Fibonacci dizisi, her sayının kendisinden önceki iki sayının toplamı olduğu bir dizidir. Bu basit algoritma, en az 1000 ondalık basamağa ulaşana kadar Fibonacci sayılarını yineler, sonra da yazdırır. Bu değer, bir `ulong` veri tipinin tutabileceğinden bile önemli ölçüde daha büyüktür. Teorik olarak, `BigInteger` sınıfındaki tek sınır, uygulamanızın tüketebileceği RAM miktarıdır;

```

using System.Numerics;
BigInteger l1 = 1;
BigInteger l2 = 1;
BigInteger current = l1 + l2;
while (current.ToString().Length < 1000)
{
    l2 = l1;
    l1 = current;
    current = l1 + l2;
}

```

```
Console.WriteLine(current);
```

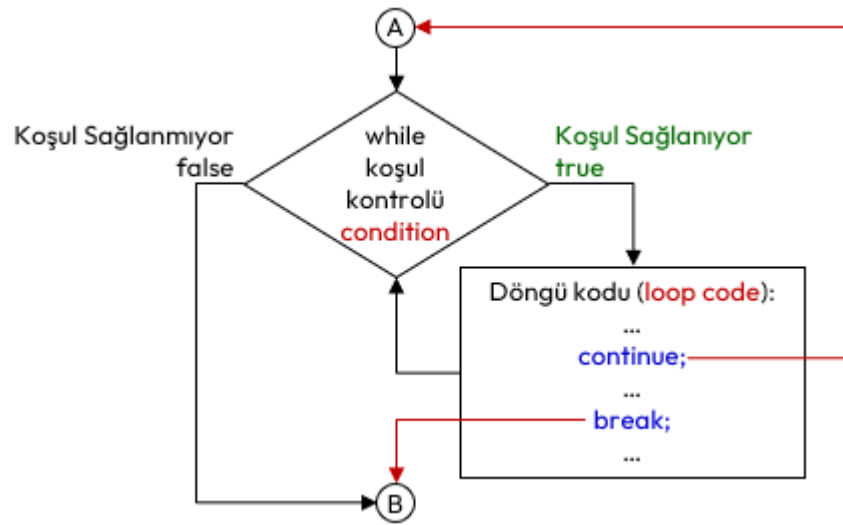
6.5 Dallanmalar

Dallanmalar (**jump**), bir döngünün yada fonksiyonun icrasını kesen ve bir başka talimata yönlendiren talimatlardır.

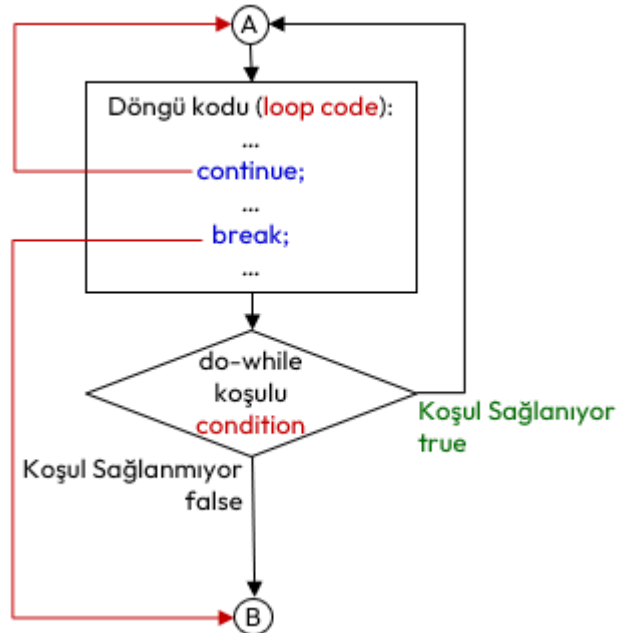
Ş6.5.1 Continue ve Break Talimatları

Continue talimatı, yalnızca geçerli yinelemeyi (**iteration**) sonlandırır ve sonraki yinelemelerle devam eder. Yani bu talimat, bir sonraki yinelemeyi yapmak için kullanılır. Dolayısıyla sadece **while**, **do-while** ve **for** döngü kodları (**loop code**) içinde kullanılabilir.

Break talimatı, döngüyü sonlandırır ve program icrası döngü sonrası ilk talimattan devam eder. Bunu daha önce **switch** talimatında görmüştük. Bu talimat, aynı şekilde **while**, **do-while** ve **for** döngüleri ile kullanılabilir. Bu talimatlarının döngü kodlarında kullanılması halinde icra akışı Şekil 6.12’de verilmiştir.



(a) While



(b) Do-While

Şekil 6.12: While ve Do-While Döngülerinde Break ve Continue Talimatları İcra Akışı

Bu talimatlar döngü kodu içinde amacımıza uygun olarak birden çok kullanabiliriz. Aşağıda 0 ile 15 arasındaki sayıların beşe bölünenleri ve 10 dışındaki sayılar ekrana yazdıran bir program örneği verilmiştir.

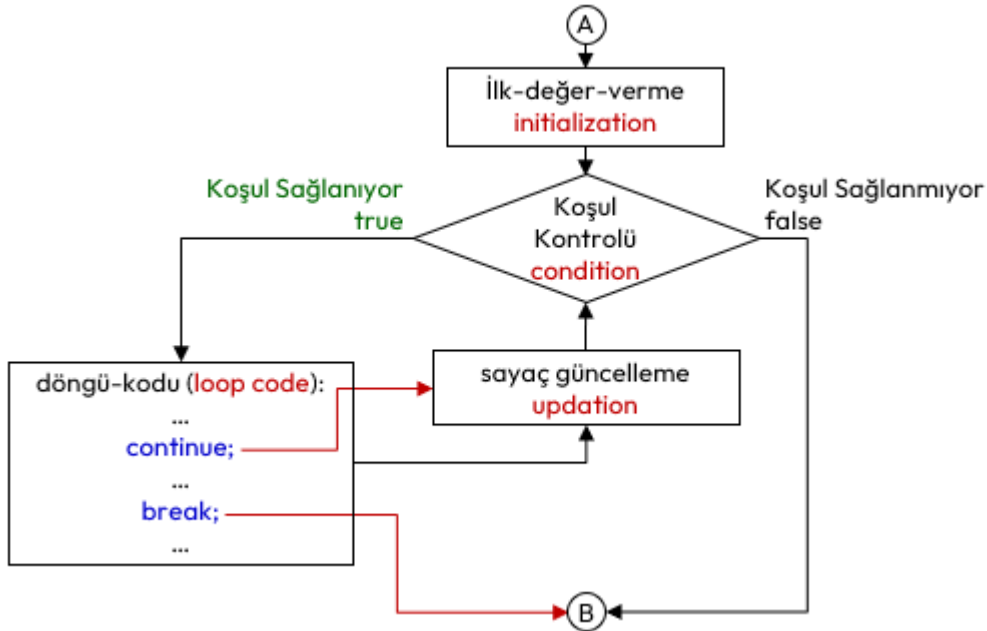
```
for (int sayaç = 0; sayaç <= 15; sayaç = sayaç + 1) {
    if (sayaç < 7) //sayaç'ın değeri 7 den küçük ise
```

```

    continue; //bir sonraki yinelemeye geç
if (sayac == 10) //sayac değeri 10 ise
    continue; //bir sonraki yinelemeye geç
if (sayac % 5 == 0) // sayac değeri 5 e bölünüyorsa
    continue; //bir sonraki yinelemeye geç
Console.WriteLine($"{sayac}");
}
/*Program Çıktısı:
7
8
9
11
12
13
14
*/

```

Yukarıdaki örnek incelendiğinde döngü bloğu üç adet talimat içeren mantıksal satır (**logical sequence**) içermesine rağmen 42 adet fiziksel satır (**physical sequence**) icra edilmiştir. Bazı `continue;` talimatları bazı satırların icrasını engellemiştir. Aşağıda pozitif sayı girilmesini zorlayan **sonsuz döngü** (**infinite loop**) içeren



Şekil 6.13: For Döngüsünde Break ve Continue Talimatları İcra Akışı

program verilmiştir. Programda döngü, pozitif sayı girildiğinde `break;` talimatı ile kırılmaktadır.

```

int sayı;
int pozitifSayı = 0;
while (true) // Sonsuz döngü başlatılıyor
{
    Console.Write("Lütfen pozitif bir tam sayı giriniz: ");
    // Güvenlik: Harf girilirse programın çökmemesi için TryParse kullanıyoruz.
    bool geçerliMi = int.TryParse(Console.ReadLine(), out sayı);
    if (geçerliMi && sayı > 0)
        break; // Şart sağlandığında döngüden zorla çıkılır.
    // Şart sağlanmazsa bu kısım çalışır ve döngü başa döner.
    Console.WriteLine("HATA: Pozitif bir tam sayı girmediniz!");
}
pozitifSayı = sayı;
Console.WriteLine($"Girilen pozitif tam sayı: {pozitifSayı}");

```

Bu talimatları dikkatli kullanmalıyız. Bazı durumlarda sonsuz döngüye girme durumu tüm döngü talimatlarında yaşanabilir. For döngüsünde bu durum farklıdır. `continue;` talimatı sonrasında bir sonraki yineleme için önce sayaç güncelleme ifadeleri icra edilir ve sonra koşul testi yapılır.

§6.5.2 Return Talimatı

Yöntemlerde çokça kullanacağımız return talimatı yapısal programlamadaki bir fonksiyonun gibidir. Yöntemler de fonksiyonlar gibi kodlanır. Üreteceği değerleri `return` talimatı ile geri döndürür.

Aşağıda üç defa negatif sayı girilmesi halinde programı sonlandıran bir klasik bir projedeki `Program.cs` örneği verilmiştir;

```
int sayı;
int pozitifSayı = 0;
int sayac = 0; // Kaç defa hatalı giriş yapıldığını tutan değişken.
do {
    Console.WriteLine("Lütfen pozitif bir sayı giriniz: ");
    // Kullanıcı girişi okunuyor ve sayıya çevriliyor.
    bool basarili = int.TryParse(Console.ReadLine(), out sayı);
    // Giriş başarısızsa veya sayı pozitif değilse hata bloğuna girer.
    if (!basarili || sayı <= 0) {
        Console.WriteLine("HATA: Pozitif Sayı GİRMEDİNİZ!");
        sayac++;
        if (sayac == 3) {
            Console.WriteLine("Üç defa hatalı veya negatif sayı girdiniz.");
            Console.WriteLine("Programdan Çıkılıyor...");

            // İşletim sistemine hata kodu (1) göndererek programı sonlandırır.
            return 1;
        }
    }
} while (sayı <= 0);
pozitifSayı = sayı;
Console.WriteLine($"Girilen pozitif tam sayı: {pozitifSayı}");
return 0; // Program başarıyla tamamlandı.

/*Program Çıktısı:
Lütfen pozitif sayı giriniz: -1
HATA: Pozitif Sayı GİRMEDİNİZ!
Lütfen pozitif sayı giriniz: -2
HATA: Pozitif Sayı GİRMEDİNİZ!
Lütfen pozitif sayı giriniz: -3
HATA: Pozitif Sayı GİRMEDİNİZ!
Üç defa negatif sayı girdiniz. Programdan Çıkılıyor.
*/
```

§6.5.3 Goto Talimatı

Tanımlı bir etikete program akışını yönlendiren `goto` talimatıdır. 6.4.1 başlığında örneği verilmiştir.

6.6 Düşün ve Çöz

- ◇ **Düşün:** Sarkan else (**dangling else**) problemi nedir? Bu belirsizliği önlemek için neden her zaman süslü parantez kullanılması önerilir? Gerçek bir projede bu hatayı bulmak neden zor olabilir?
- ◇ **Düşün:** While ve Do-while döngüleri arasındaki farkı, ATM’de PIN kodu doğrulama senaryosu üzerinden karşılaştırınız. Hangi senaryoda do-while daha doğal bir tercih olur?
- ◇ **Düşün:** Switch ifadesinde `break`; kullanılmaması durumundaki ‘fall-through’ davranışını hem bir hata hem de kasıtlı bir özellik olarak kullanmak mümkün müdür? C#8 ile gelen **switch ifadesi (expression)** bu sorunu nasıl ele alır?
- ◇ **Düşün:** `for` döngüsünde döngü değişkeninin döngü dışında değiştirilmesi neden tehlikelidir? Sonsuz döngü yazmanın kasıtlı ve kasıtsız yolları nelerdir? Sonsuz döngülerin gerçekte kullanıldığı senaryolara örnek veriniz.
- ◇ **Düşün:** `goto` talimatı neden ‘zararlı’ kabul edilir? Bu talimatın gerçekten faydalı olduğu (ya da kaçınılmaz görüldüğü) bir durum düşünebilir misiniz? C#’ta `goto` kullanımı hangi durumda mantıklı olabilir?
- ◇ **Düşün:** Döngü içinde `continue` ve `break` talimatlarının yanlış kullanımı ne tür mantık hatalarına yol açabilir? İç içe döngülerde `break`’in yalnızca içteki döngüden çıkması bazen istenmedik bir davranışa neden olur: bunu nasıl çözürsünüz?
- ◇ **Düşün:** Koşul ifadelerinde kısa devre değerlendirmesi (**short-circuit evaluation**) nedir? `&&` ve `&` opera-

törleri arasındaki farkı bir `null` kontrolü senaryosuyla açıklayınız.

- ◇ **Çöz:** Kullanıcıdan sürekli tamsayı girişi alan; 0 girildiğinde durarak, o ana kadar girilen sayıların en büyüğünü, en küçüğünü, toplamını ve ortalamasını ekrana yazan bir gözcü kontrollü döngü tasarlayınız.
- ◇ **Çöz:** 1'den 100'e kadar olan sayıları ekrana yazan, ancak 3'ün katları için sayı yerine 'Fizz', 5'in katları için 'Buzz', her ikisinin de katları için 'FizzBuzz' yazan programı yazınız (klasik FizzBuzz).
- ◇ **Çöz:** Kullanıcıdan bir tam sayı alarak o sayının asal olup olmadığını belirleyen programı yazınız. Ardından 2 ile 100 arasındaki tüm asal sayıları listeleyen (Eratosthenes Kalburu) versiyonunu geliştiriniz.
- ◇ **Çöz:** İç içe döngülerle 1'den N'e kadar olan sayıların çarpım tablosunu (N kullanıcıdan alınacak) düzgün sütun hizalamasıyla ekrana basan programı yazınız.
- ◇ **Çöz:** Bir ATM şifre ekranı simüle edin: kullanıcı doğru 4 haneli PIN'i (sabit olarak tanımlanmış) girinceye kadar en fazla 3 deneme hakkı verilsin; 3 denemede de yanlış girilirse kart bloke mesajı gösterilsin.
- ◇ **Düşün :** Bir `if` bloğu içinde `(x > 0 | y++ > 0)` yazmak ile `(x > 0 || y++ > 0)` yazmak arasında, `y` değişkeninin son değeri açısından nasıl bir fark oluşur?
- ◇ **Düşün :** Yüzlerce koşulun olduğu bir senaryoda `if-else` if yapısı yerine neden bir `Dictionary` veya `switch` yapısı kullanmayı tercih etmeliyiz?